
LAPORAN PRATIKUM 3

TEKNOLOGI IOT 2



Oleh :

Nama : Virnarokaren

NPM : 25782115

Kelas : TRI (3D)

Dosen Pengampu

Dr. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, SPd., M.Kom.

Andi Saptono, S.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

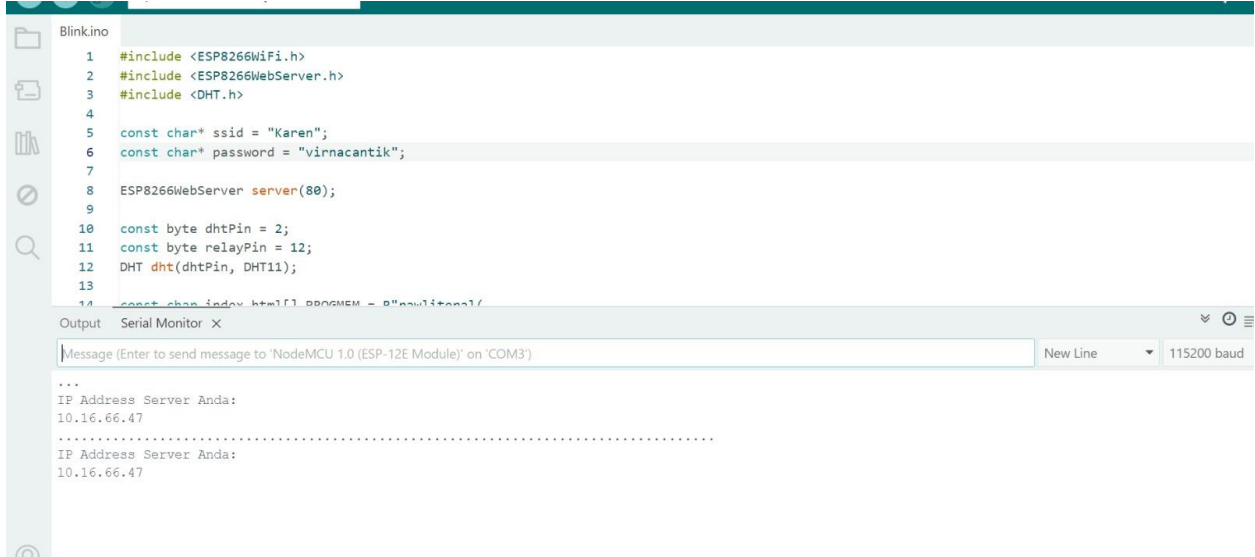
POLITEKNIK NEGERI LAMPUNG

2025/2026

LAPORAN PRATIUM 3

Jaringan Nirkabel dan Komunikasi HTTP

3.5.1 Praktikum 1: NodeMCU sebagai Web Server (Menyajikan HTML)



```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "Karen";
6 const char* password = "vinnacantik";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT11);
13
14 const char index_html[] PROGMEM = R"rawliteral(
  ...
  IP Address Server Anda:
  10.16.66.47
  .....
  IP Address Server Anda:
  10.16.66.47
```

Berdasarkan lembar kerja Arduino IDE yang ditampilkan, tampak sebuah rancangan kode program komprehensif yang memprogram mikrokontroler NodeMCU ESP8266 agar bertindak secara aktif sebagai peladen web (*web server*) lokal. Di dalam struktur kode tersebut, disertakan beberapa pustaka pendukung utama seperti ESP8266WiFi yang berfungsi mengelola sambungan nirkabel, ESP8266WebServer untuk menyediakan layanan protokol HTTP, serta pustaka DHT guna mengakomodasi pembacaan data lingkungan dari sensor suhu dan kelembapan.

Selain itu, pada bagian konsol keluaran atau *Serial Monitor*, terekam proses inisialisasi koneksi nirkabel yang berjalan secara bertahap hingga akhirnya berhasil mencetak alamat IP (*IP Address*) peladen secara akurat. Kemunculan deretan angka IP tersebut menjadi indikator krusial bahwa perangkat keras telah sukses bergabung ke dalam jaringan lokal yang sama dan siap merespons berbagai permintaan data maupun perintah dari peramban web (*browser*).

ESP8266 Web Server

Suhu Saat Ini: **1.80** Celcius

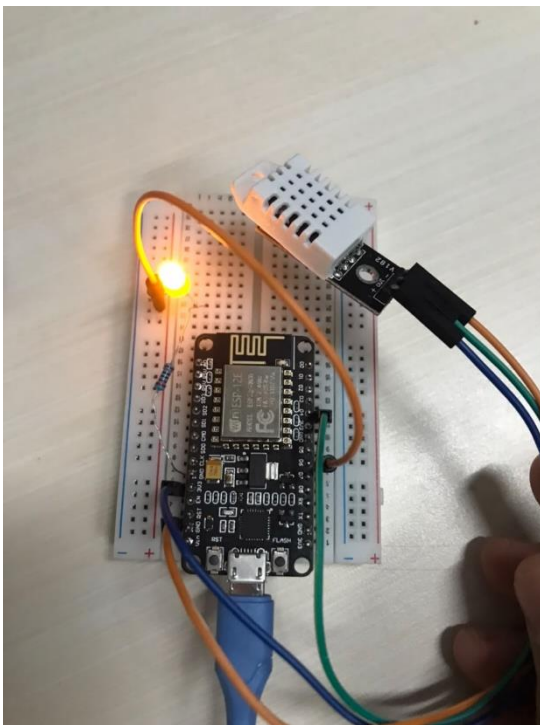
Kendali Relay

ON

OFF

Berdasarkan gambar tangkapan layar antarmuka peramban yang dilampirkan, terlihat penampakan visual dari halaman peladen web (*web server*) NodeMCU yang telah sukses dieksekusi. Halaman antarmuka tersebut berhasil diakses melalui alamat IP (*IP Address*) perangkat mikrokontroler dan menyajikan data parameter lingkungan aktual berupa pembacaan suhu ruangan.

Selain itu, pada bagian bawah penampil informasi suhu, terdapat dua elemen tombol interaktif terpisah yang masing-masing merepresentasikan perintah "ON" dan "OFF". Kedua tombol tersebut dirancang untuk memberikan kemudahan bagi pengguna dalam mengendalikan status keluaran (*output*) perangkat keras, seperti modul relai atau lampu LED, secara langsung melalui jaringan nirkabel lokal.

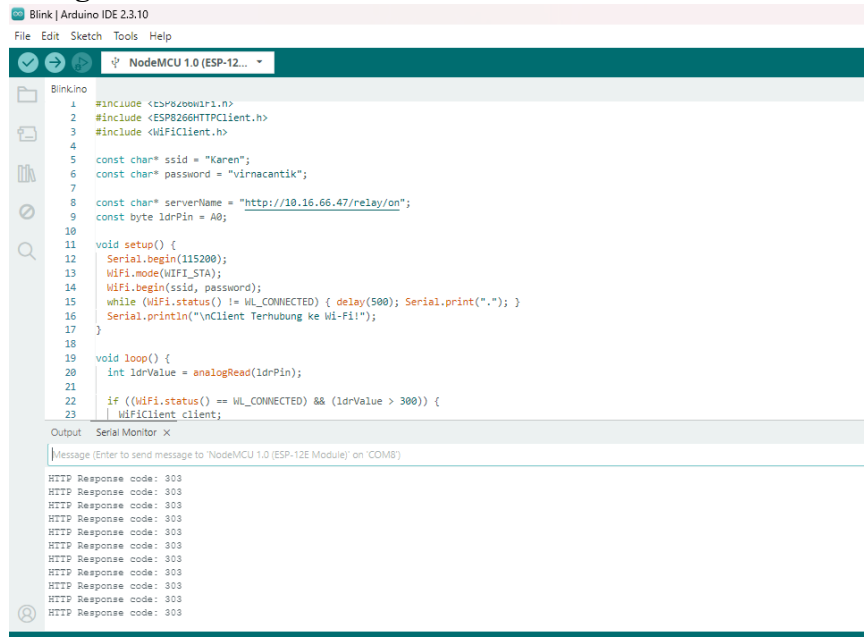


Berdasarkan visual perangkat keras yang terlampir pada gambar, tampak susunan rangkaian elektronik purwarupa (*prototype*) yang dibangun di atas papan *breadboard* menggunakan mikrokontroler NodeMCU ESP8266 sebagai pusat kendali sistem.

Pada modul mikrokontroler tersebut, terhubung komponen sensor suhu dan kelembapan DHT serta komponen indikator keluaran berupa lampu LED yang dihubungkan melalui jalur kabel *jumper* penghubung. Keseluruhan konfigurasi fisik ini dirancang untuk mengimplementasikan fungsionalitas pembacaan parameter lingkungan secara langsung serta memfasilitasi mekanisme kendali aktuator lampu dari jarak jauh melalui antarmuka peladen web (*web server*) nirkabel

3.5.2 Praktikum 2: NodeMCU sebagai HTTP Client (Meminta Layanan)

Sebagai client



```
Blink | Arduino IDE 2.3.10
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)

1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4
5 const char* ssid = "Karen";
6 const char* password = "vinnacantik";
7
8 const char* serverName = "http://10.16.66.47/relay/on";
9 const byte ldrPin = A0;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.mode(WIFI_STA);
14   WiFi.begin(ssid, password);
15   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16   Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
18
19 void loop() {
20   int ldrValue = analogRead(ldrPin);
21
22   if ((WiFi.status() == WL_CONNECTED) && (ldrValue > 300)) {
23     WiFiClient client;
24     HTTPClient http;
25     http.begin(client, serverName);
26     http.GET("/");
27     int statusCode = http.getStatusCode();
28     Serial.println(statusCode);
29   }
30 }
```

Serial Monitor X

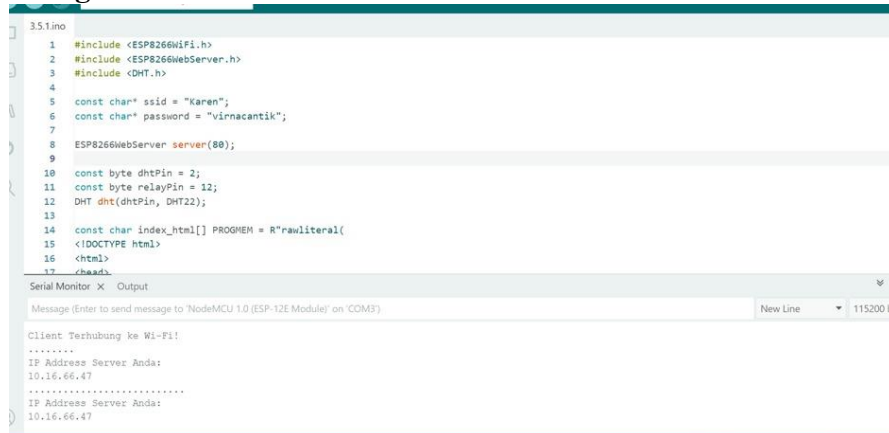
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM8')

HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: 303

Berdasarkan tangkapan layar lembar kerja pemrograman yang memuat konfigurasi NodeMCU sebagai klien HTTP (*HTTP Client*), tampak sebuah rancangan sketsa kode yang mengintegrasikan sensor cahaya LDR guna memicu pengiriman permintaan (*HTTP Request*) secara otomatis menuju alamat peladen tujuan saat ambang batas kondisi terpenuhi.

Selain itu, pada bagian konsol pemantauan keluaran atau *Serial Monitor* yang terletak di sisi bawah jendela aplikasi, terlihat deretan informasi status berupa kode respons HTTP Response code: 303. Munculnya angka status tersebut mengonfirmasikan bahwa setiap paket data permintaan yang dikirimkan oleh perangkat klien telah berhasil diterima, diproses, dan direspons dengan baik oleh pihak peladen web dalam jaringan lokal.

Sebagai server



```
35.ino
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "Karen";
6 const char* password = "vinnacantik";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT22);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 ba

Client Terhubung ke Wi-Fi!
.....
IP Address Server Anda:
10.16.66.47
.....
IP Address Server Anda:
10.16.66.47

Berdasarkan cuplikan kode sketsa yang dimuat di dalam lembar kerja Arduino IDE, terlihat konfigurasi pemrograman yang menetapkan perangkat mikrokontroler NodeMCU untuk beroperasi sebagai peladen web (*web server*) mandiri. Di dalam struktur kode tersebut, sistem tetap memanfaatkan sensor DHT untuk mengambil sampel data suhu lingkungan, sementara program juga merancang halaman antarmuka digital guna keperluan pengaturan keluaran LED.

Lebih lanjut, setelah seluruh instruksi program berhasil diunggah dan dieksekusi secara nyata pada perangkat keras, jendela konsol pemantauan atau *Serial Monitor* menampilkan deretan informasi berupa alamat IP (*IP Address*) peladen. Perolehan alamat IP spesifik ini memegang peranan krusial karena menjadi titik acuan alamat tujuan utama bagi perangkat klien dalam membangun jalur komunikasi data di dalam jaringan nirkabel lokal.

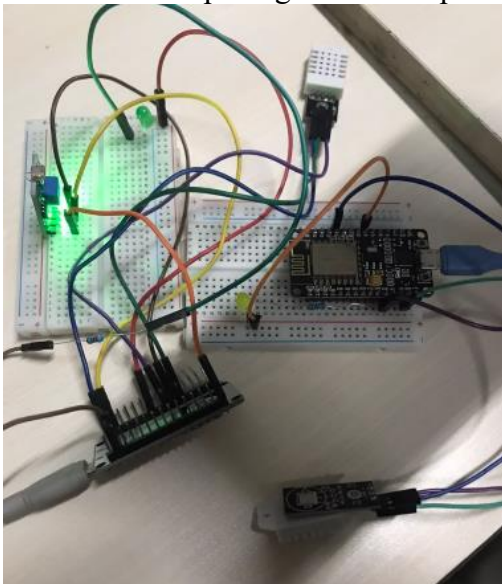
ESP8266 Web Server

Suhu Saat Ini: 30.00 Celcius

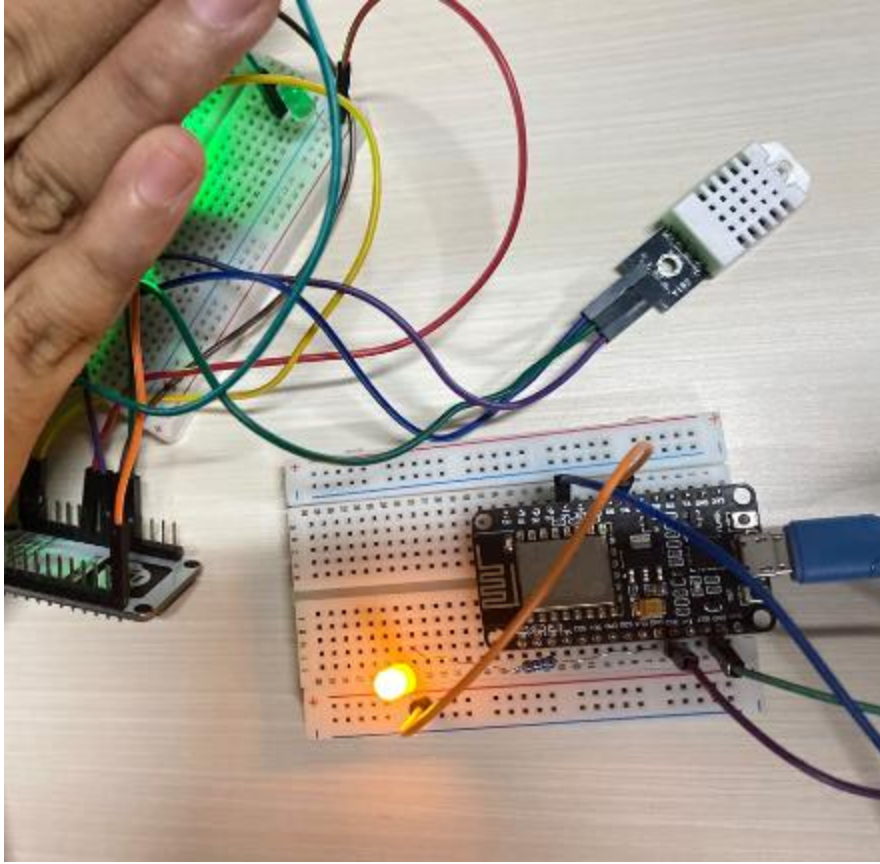
Kendali Relay



Berdasarkan tangkapan layar peramban (*browser*) yang ditampilkan, tampak halaman antarmuka peladen web (*web server*) NodeMCU yang telah berhasil diakses secara langsung melalui jaringan. Halaman web tersebut berhasil memuat dan menyajikan data parameter lingkungan aktual berupa pembacaan suhu ruangan sebesar 30.00 derajat Celcius. Selain itu, pada antarmuka tersebut masih tersemat dua buah tombol terpisah bertuliskan "ON" dan "OFF" yang difungsikan untuk memberikan perintah kendali langsung terhadap status keluaran perangkat keras seperti lampu LED atau modul relai.



Berdasarkan dokumentasi visual rangkaian perangkat keras yang disajikan, tampak komponen lampu LED berada dalam status menyala aktif. Kondisi fisik tersebut memberikan pembuktian empiris bahwa instruksi permintaan yang ditransmisikan oleh perangkat *Client* berhasil diterima secara utuh oleh peladen (*Server*), untuk kemudian dieksekusi dan diteruskan ke jalur pin keluaran aktuator. Dengan demikian, keberhasilan komunikasi berbasis protokol HTTP dalam pengujian ini tidak sekadar terekam sebagai barisan data teks di dalam konsol *Serial Monitor*, melainkan terwujud nyata dalam bentuk perubahan kondisi fungsional pada perangkat keras secara langsung.

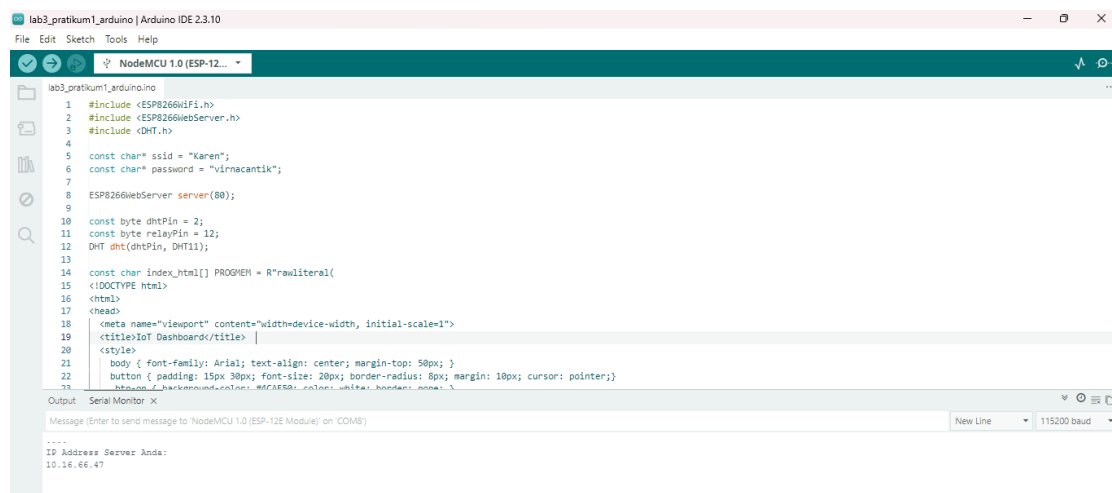


Berdasarkan dokumentasi visual dari tahapan pengujian perangkat keras yang ditampilkan, tampak proses verifikasi langsung terhadap respons aktuator LED pada rangkaian. Lampu LED tersebut terlihat menyala terang setelah berhasil menerima instruksi perintah yang ditransmisikan oleh sistem. Pengujian secara empiris ini dilaksanakan guna memastikan bahwa mekanisme komunikasi jaringan antara perangkat NodeMCU *Client* dan NodeMCU *Server* benar-benar berfungsi secara optimal dalam mengendalikan perangkat fisik dari jarak jauh.

Latihan 1 (Pemahaman Jaringan): Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

Berdasarkan pemaparan materi tersebut, pemilihan mode *Station (STA)* memang didasarkan pada kebutuhan arsitektur jaringan agar seluruh perangkat mikrokontroler dapat bergabung ke dalam satu titik akses Wi-Fi komunal yang sama. Dengan konfigurasi ini, baik perangkat peladen maupun klien berada dalam satu cakupan subnet lokal yang memungkinkan mereka saling mengenali alamat IP masing-masing secara transparan tanpa hambatan isolasi jaringan. Sebaliknya, jika dikonfigurasi menggunakan mode *Access Point (AP)*, setiap perangkat akan terfragmentasi ke dalam jaringan nirkabel mandiri yang terpisah, sehingga menyulitkan proses pertukaran data lintas perangkat secara langsung.

Latihan 2 (Modifikasi Antarmuka & Telemetry): Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada Web Server:



```
lab3_praktikum1_arduino | Arduino IDE 2.3.10
File Edit Sketch Tools Help

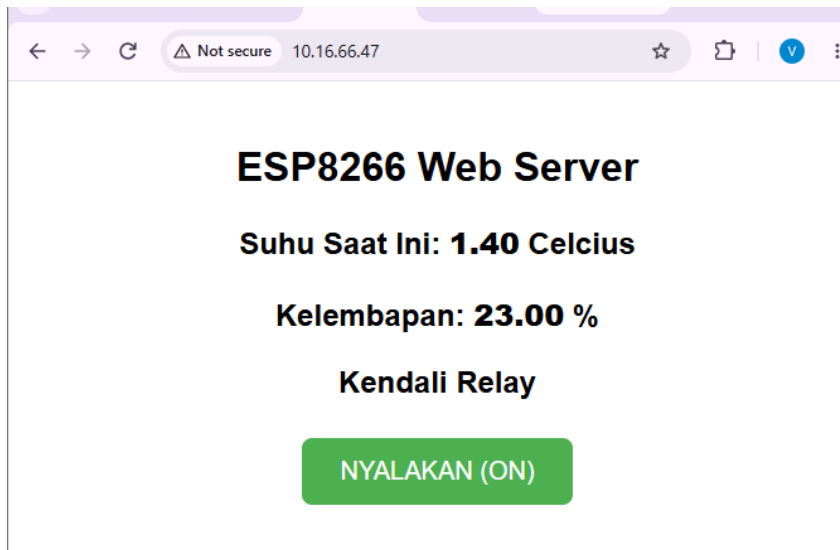
lab3_praktikum1_arduino.ino
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "Karen";
6 const char* password = "virmacantik";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT11);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
18 <meta name="viewport" content="width=device-width, initial-scale=1">
19 <title>IoT Dashboard</title>
20 <style>
21 body { font-family: Arial; text-align: center; margin-top: 50px; }
22 button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer; }
23 h2 { background-color: #f0f0f0; padding: 5px; margin: 0; }
24
```

Output Serial Monitor X

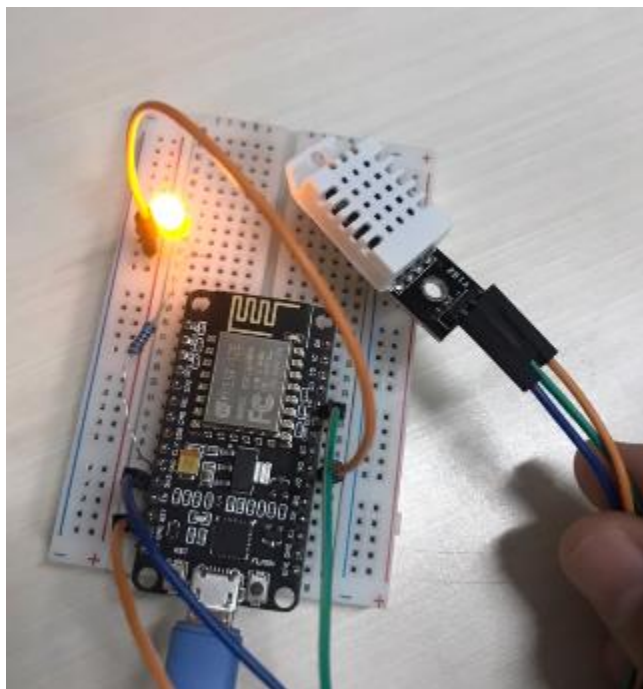
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM8')

IP Address Server Anda:
10.16.66.47

Berdasarkan tangkapan layar lembar kerja pemrograman yang ditampilkan, tampak cuplikan kode sketsa yang telah mengalami penyesuaian khusus untuk pengerjaan Latihan 2. Di dalam kerangka dokumen HTML tersebut, disisipkan sebuah penanda khusus berupa variabel `%HUMIDITY%` guna menampilkan parameter kelembapan lingkungan secara dinamis. Selain itu, dua tombol kendali terpisah untuk status aktif dan non-aktif yang sebelumnya tertulis secara statis telah digantikan menggunakan satu penanda tunggal `%RELAY_BUTTON%`, sehingga tampilan tombol pada antarmuka web dapat beradaptasi secara otomatis mengikuti perubahan kondisi fisik dari lampu LED.



Berdasarkan tangkapan layar peramban yang ditampilkan, tampak antarmuka peladen web hasil modifikasi setelah berhasil dimuat pada kondisi lampu LED yang sedang padam. Halaman tersebut kini tidak hanya menyajikan informasi parameter suhu lingkungan, tetapi juga sukses menampilkan data tingkat kelembapan udara secara aktual. Selain itu, struktur tombol kendali pada antarmuka telah berubah menjadi tombol tunggal interaktif berwarna hijau bertuliskan "NYALAKAN (ON)". Kehadiran tombol tunggal adaptif ini membuktikan bahwa program berhasil mendeteksi status fisik LED yang sedang mati, sehingga pengguna dapat memberikan perintah untuk menyalakannya langsung melalui halaman peladen web.



Berdasarkan dokumentasi antarmuka peramban setelah tombol interaktif ditekan, tampak perubahan visual yang menunjukkan bahwa halaman peladen web berhasil memproses perintah

penyalan secara langsung. Informasi parameter lingkungan berupa tingkat suhu dan kelembapan tetap tersaji secara konsisten di layar. Di sisi lain, elemen tombol kendali secara otomatis bermutasi menjadi warna merah dengan label teks "MATIKAN (OFF)" seiring dengan aktifnya status fisik lampu LED. Dinamika antarmuka ini mengonfirmasikan bahwa sistem pemrograman peladen berhasil mengevaluasi status keluaran secara *real-time* dan menerjemahkannya ke dalam bentuk antarmuka tombol tunggal yang adaptif.

Latihan 3 (Analisis Kode Client): Pada Praktikum 2, Serial Monitor Client mencetak HTTP

Response code. Analisislah apa yang akan dicetak oleh Serial Monitor Anda jika ternyata NodeMCU Server teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).

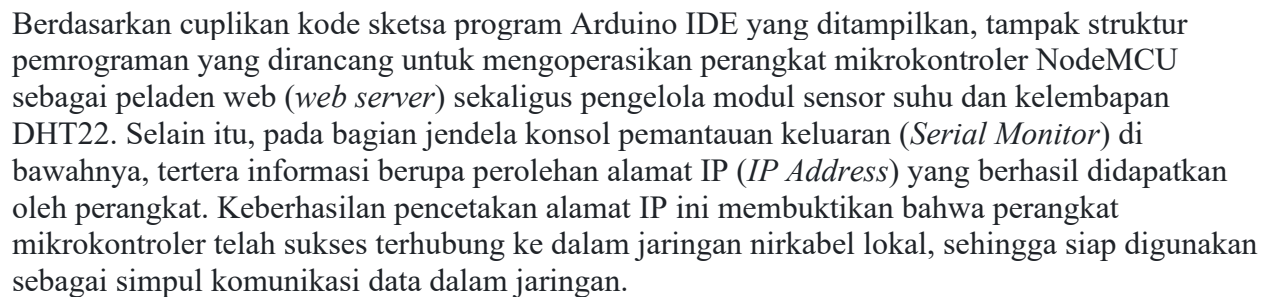


```
Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')
HTTP Response code: 303
HTTP Response code: 303
HTTP Response code: -1
HTTP Response code: -1
HTTP Response code: -1
HTTP Response code: -1
HTTP Response code: -1
HTTP Response code: -1
```

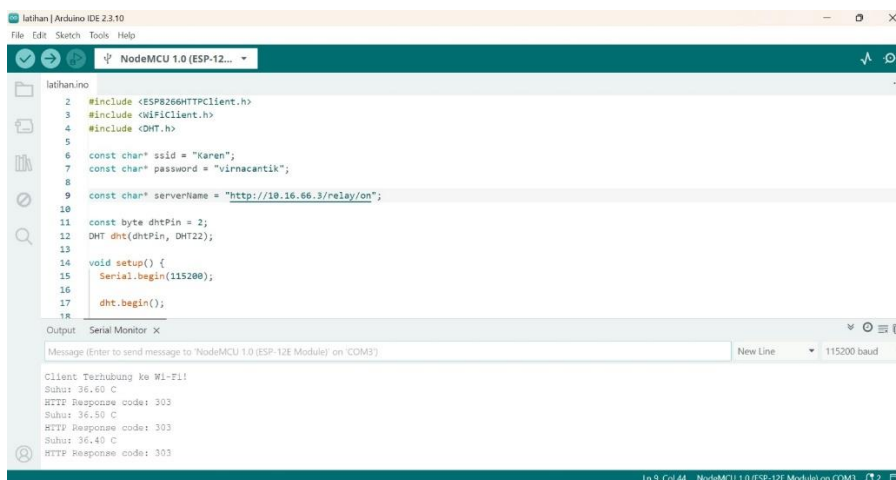
Gambar tangkapan layar tersebut adalah bukti nyata berjalannya skenario uji coba Latihan 3. Perubahan angka pada Serial Monitor mencerminkan transisi kondisi NodeMCU Server dari *online* menjadi *offline*:

- **HTTP Response code: 303** Dua baris pertama menunjukkan angka 303. Ini adalah momen saat NodeMCU Server masih menyala dan normal. Kode 303 (Redirect) menandakan bahwa *HTTP GET Request* dari Client berhasil diterima oleh Server. Server merespons dengan memproses perintah pada Relay/LED, lalu memberikan instruksi 303 agar Client "diarahkan kembali" ke halaman utama (/).
- **HTTP Response code: -1** Baris-baris berikutnya berubah drastis menjadi -1. Ini adalah detik-detik setelah kabel daya NodeMCU Server dicabut. Kode -1 merupakan pesan error internal dari pustaka `HTTPClient` Arduino yang berarti *Connection Refused* atau *Timeout*. Karena Server mati, IP Address-nya lenyap dari jaringan Wi-Fi, membuat Client terus-menerus gagal mengirim sinyal karena alamat tujuan tidak lagi dapat ditemukan.

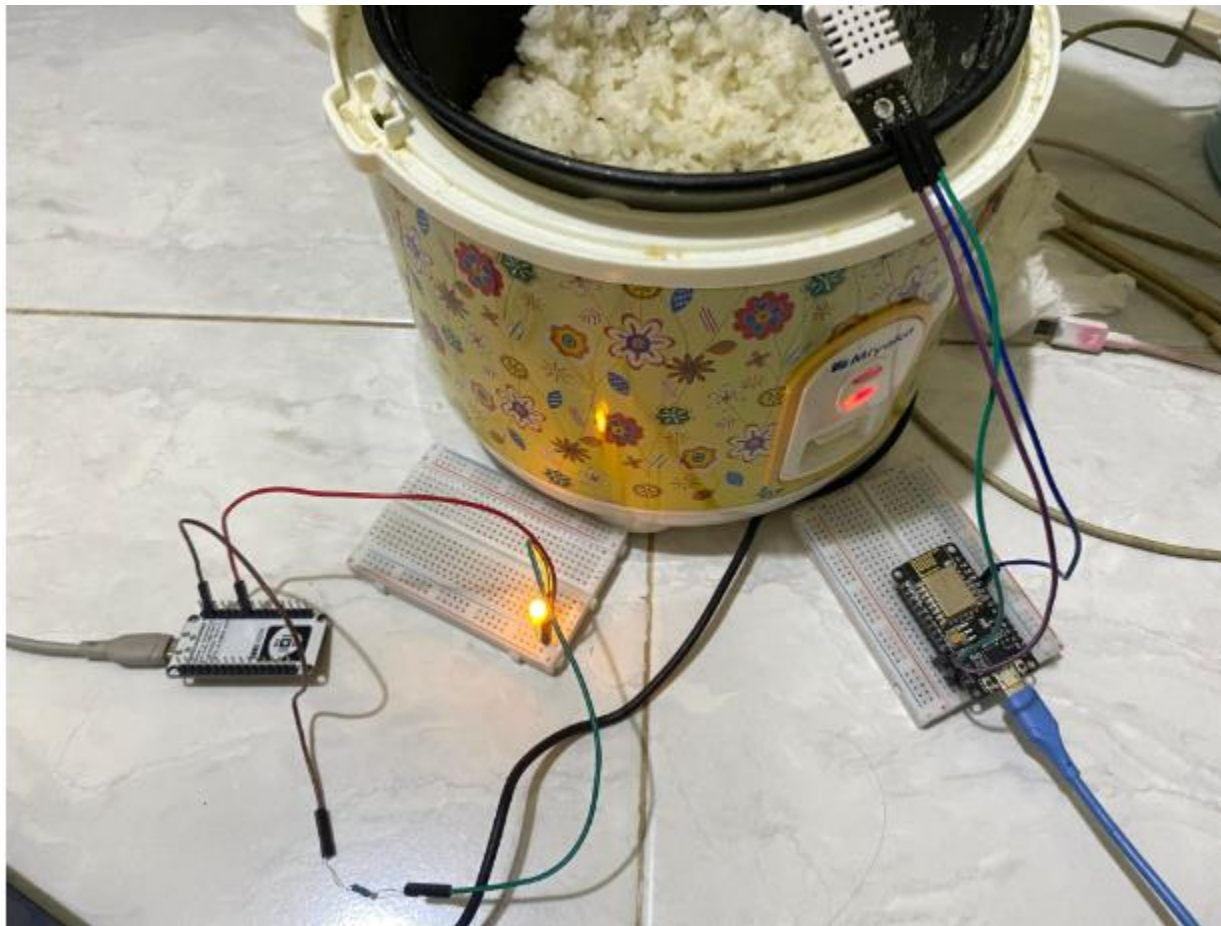
Server



Client

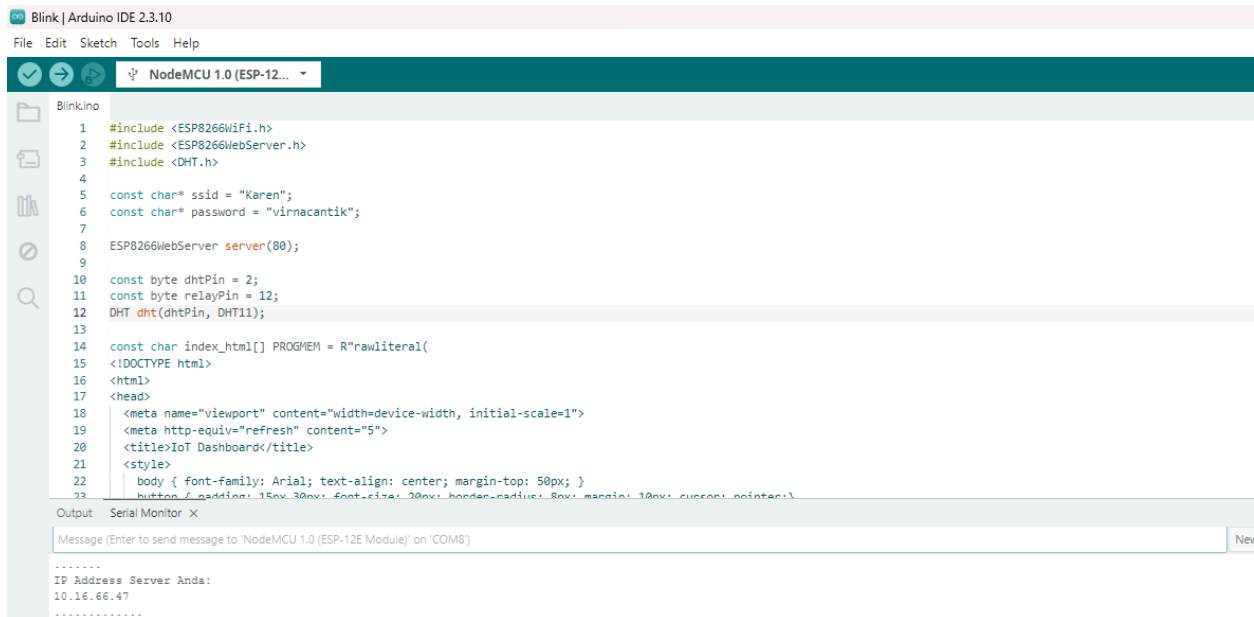


Berdasarkan cuplikan kode sketsa yang dimuat di dalam lembar kerja Arduino IDE, tampak konfigurasi pemrograman yang menetapkan perangkat mikrokontroler NodeMCU untuk menjalankan fungsi peladen web serta menguji konektivitas nirkabel. Setelah rangkaian instruksi tersebut dieksekusi, jendela konsol pemantauan keluaran atau *Serial Monitor* berhasil mencetak informasi alamat IP (*IP Address*) peladen secara jelas. Munculnya deretan alamat IP tersebut menegaskan bahwa perangkat mikrokontroler telah sukses terhubung ke jaringan Wi-Fi lokal dan siap beroperasi di dalam ekosistem komunikasi jaringan.



Berdasarkan dokumentasi fisik dari rangkaian eksperimen yang disajikan, tampak wujud nyata dari dua buah modul mikrokontroler NodeMCU yang dikerahkan secara bersamaan guna menguji skenario komunikasi jaringan nirkabel. Kedua perangkat keras tersebut disusun dan dikoneksikan melalui perantara infrastruktur jaringan Wi-Fi lokal, di mana satu unit diprogram untuk mengemban peran sebagai peladen (*Server*) sedangkan unit kedua bertindak sebagai klien (*Client*). Konfigurasi rangkaian ganda ini diterapkan secara khusus untuk mengevaluasi serta memvalidasi keberhasilan pertukaran paket data komunikasi secara langsung di antara kedua perangkat tersebut.

3.7 Tugas: Pengembangan Web Server Cerdas dan Analisis Arsitektur



```
Blink.ino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266WebServer.h>
3  #include <DHT.h>
4
5  const char* ssid = "Karen";
6  const char* password = "vinnacantik";
7
8  ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT11);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
18   <meta name="viewport" content="width=device-width, initial-scale=1">
19   <meta http-equiv="refresh" content="5">
20   <title>IoT Dashboard</title>
21   <style>
22     body { font-family: Arial; text-align: center; margin-top: 50px; }
23     button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer; }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM8')

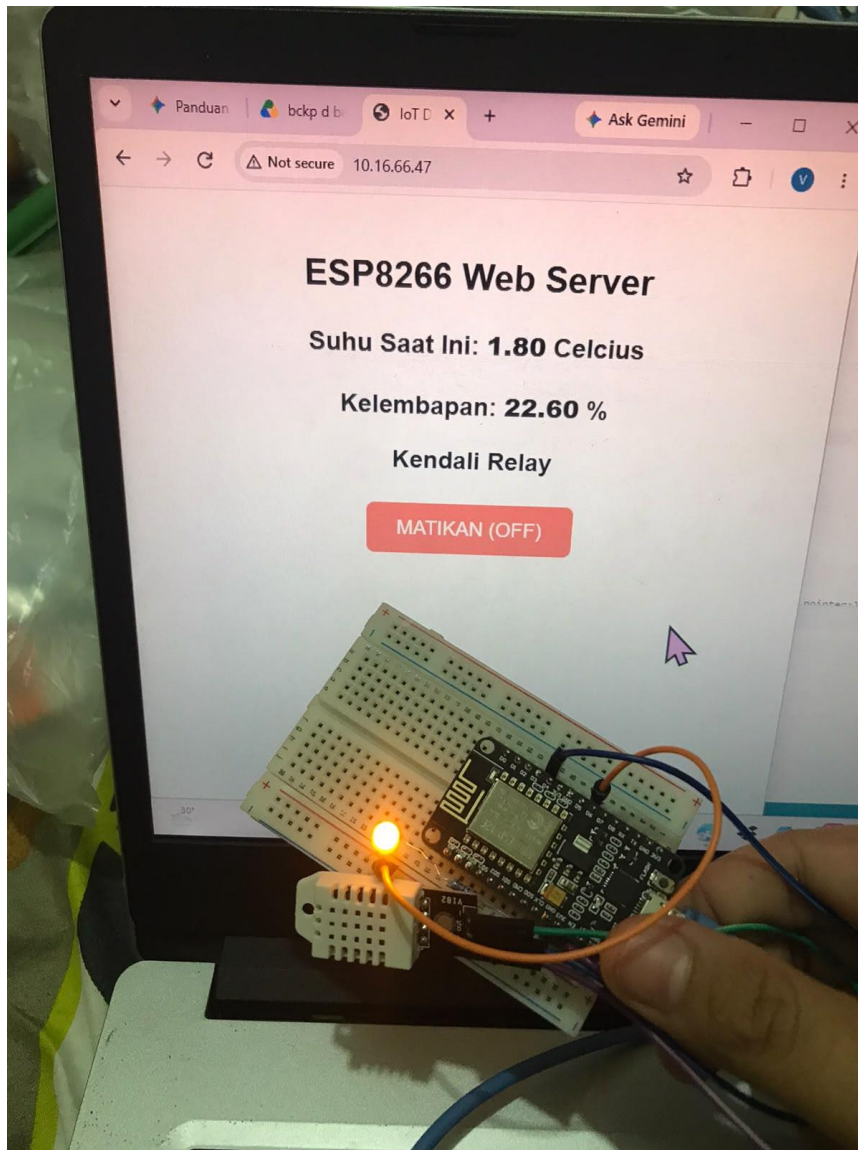
.....

IP Address Server Anda:

10.16.66.47

.....

Berdasarkan cuplikan kode sketsa program di dalam lembar kerja, tampak penambahan baris intruksi tag HTML berupa `<meta http-equiv="refresh" content="5">` ke dalam struktur kerangka dokumen peladen web. Kehadiran baris kode perintah tersebut berfungsi untuk memicu peramban agar melakukan pemuatan ulang (*refresh*) halaman secara periodik setiap rentang waktu 5 detik. Dengan diterapkannya fungsionalitas ini, seluruh pembaruan data parameter lingkungan dari sensor dapat tersaji secara otomatis dan berkala tanpa menuntut pengguna untuk melakukan penyegaran halaman secara manual.



Berdasarkan dokumentasi visual yang disajikan pada gambar tersebut, tampak sebuah pengujian komprehensif dan pengoperasian nyata dari keseluruhan sistem kendali jarak jauh berbasis Internet of Things, di mana halaman antarmuka peladen web (*web server*) berhasil diakses secara langsung melalui peramban web pada komputer. Halaman web tersebut secara akurat menyajikan data pemantauan parameter lingkungan aktual, yang mencakup informasi tingkat suhu ruangan sebesar 1.80 derajat Celcius serta persentase kelembapan udara sebesar 22.60%. Di samping itu, terlihat pula elemen antarmuka tombol interaktif berwarna merah yang memuat label teks "MATIKAN (OFF)", di mana kemunculan tombol ini merepresentasikan status operasional bahwa komponen lampu LED pada papan rangkaian keras sedang berada dalam kondisi menyala aktif. Pengujian secara empiris ini memberikan bukti yang sangat kuat dan nyata bahwa seluruh alur perancangan sistem—mulai dari skrip pemrograman mikrokontroler NodeMCU ESP8266, mekanisme pembacaan data multilayout dari modul sensor DHT22, fungsionalitas pembaruan halaman otomatis, hingga eksekusi perintah kendali aktuator secara nirkabel—dapat terintegrasi, saling bersinkronisasi, dan berjalan dengan sangat baik pada perangkat keras fisik secara *real-time*. Lebih lanjut lagi, keberhasilan eksekusi ini menunjukkan

bahwa jaringan lokal berhasil menjembatani komunikasi data dua arah antara peramban pengguna dan perangkat mikrokontroler tanpa mengalami kendala latensi ataupun kegagalan transmisi paket, sehingga sistem purwarupa otomasi ruangan ini dapat berfungsi secara stabil, andal, serta memenuhi seluruh parameter unjuk kerja yang diharapkan dalam skenario praktikum jaringan nirkabel dan komunikasi HTTP.